

A

Thesis On

**EMERGENCY ALERTING WHEELCHAIR FOR DISABLED PEOPLE
WITH MULTIPLE CONTROL OPTIONS**

Submitted

In the Partial Fulfilment of the Requirements
for the degree of

BACHELOR OF TECHNOLOGY

By

ANU SHARMA (2000040310006)

ARZOO GOYAL (2000040310008)

PAYAL MOHNANI (2000040310032)

Under the Supervision of

Er. DHANANJALI SINGH

(Asst. Professor, ECE Department)



to the

PG Department of Electronics & Communication Engineering

RAJA BALWANT SINGH ENGINEERING TECHNICAL CAMPUS

BICHPURI, AGRA

May, 2024

DECLARATION

We hereby declare that this submission is our own work and that, to the best of our knowledge and belief, it contains no material previously published or written by another person nor material which to a substantial extent has been accepted for the award of any other degree or diploma of the university or other institute of higher learning, except where due acknowledgment has been made in the text.

S.No.	Student Name	Roll No.	Signature
1	ANU SHARMA	2000040310006	
2	ARZOO GOYAL	2000040310008	
3	PAYAL MOHNANI	2000040310032	

CERTIFICATE

Certified that **ANU SHARMA (2000040310006), ARZOO GOYAL (2000040310008)** and **PAYAL MOHNANI (2000040310032)** has carried out the project work presented in this thesis entitled “**EMERGENCY ALERTING WHEELCHAIR FOR DISABLED PEOPLE WITH MULTIPLE CONTROL OPTIONS**” for the award of Bachelor of Technology from Dr.A.P.J.Abdul Kalam Technical University, Lucknow, Uttar Pradesh, under my supervision. The thesis embodies results of original work, and studies as are carried out by the students themselves and the contents of the thesis do not form the basis for the award of any other degree to the candidate or to anybody else from this or any other University/Institution.

Er. DHANANJALI SINGH

Assistant Professor

PG Department of ECE

RBSETC, Bichpuri, Agra

Place: Agra

Date:

ABSTRACT

This project introduces an innovative wheelchair system designed to enhance independent mobility for individuals with various physical impairments, including paraplegia, locomotive syndrome, lower extremity fractures, and quadriplegia, provided they have the ability to speak. The wheelchair is equipped with dual control options: voice command and joystick, offering users flexibility and ease of use. At the core of this system is the Arduino Nano, which functions as the main controller. Voice commands are processed through a USB microphone connected to a Raspberry Pi, utilizing Google APIs for efficient and accurate speech-to-text conversion with minimal delay. The voice command feature supports basic English commands such as 'Forward', 'Backward', 'Left', 'Right', 'Stop', and 'Help', as well as simple Hindi commands like 'Aage', 'Pichhe', 'Ruko', and 'Madad', making it accessible to a broader user base.

Furthermore, the wheelchair includes a critical emergency alert feature. It employs accelerometer and gyroscope sensors to detect falls and monitor the wheelchair's orientation. In case of an accident, the system uses GPS and GSM modules to send immediate alerts and help messages to caregivers, including the current location of the wheelchair. This allows caregivers to track the wheelchair and respond promptly to emergencies. Overall, this system significantly improves the independence and safety of individuals with mobility challenges.

ACKNOWLEDGEMENTS

We would like to express our sincere gratitude to the Council of Science and Technology, Uttar Pradesh (CST UP), for their generous funding and support of this research project. Without their financial assistance, this work would not have been possible.

We also extend our thanks to Er. Dhananjali Singh, Assistant Professor, for her invaluable guidance, mentorship, and support throughout the duration of this project. Her expertise and encouragement have been instrumental in our project.

We are also indebted to the contribution of the faculty members of the PG Department of Electronics and Communication Engineering for their valuable feedback and suggestions during this project. Their insights have enriched the quality of our work.

Finally, we acknowledge our friends for their unwavering support and encouragement. Their contributions, whether through technical assistance, moral support, or constructive feedback, have been invaluable in the completion of this project.

Anu Sharma

Arzoo Goyal

Payal Mohnani

TABLE OF CONTENT

DECLARATION.....	II
CERTIFICATE.....	III
ABSTRACT.....	IV
ACKNOWLEDGEMENTS	V
LIST OF TABLES	VIII
LIST OF FIGURES	IX
CHAPTER-1 INTRODUCTION.....	1-3
1.1 PROJECT OVERVIEW.....	1
1.2 OBJECTIVES OF THIS PROJECT	2
1.3 NEED OF THIS PROJECT	3
CHAPTER-2 LITERATURE REVIEW.....	4-9
CHAPTER-3 COMPONENT DESCRIPTION	9-31
3.1 HARDWARE DESCRIPTION	9
3.1.1 Raspberry Pi 4B	9
3.1.2 Arduino Nano	11
3.1.3 SIM800L GSM Module.....	14
3.1.4 NEO 6M GPS Module.....	15
3.1.5 Joystick Module	16
3.1.6 USB Microphone	18
3.1.7 DC Motor.....	19
3.1.8 L298 Motor Driver Module	20
3.1.9 MPU6050.....	21
3.1.10 Battery.....	22
3.1.11 12V to 5V Type A Buck Converter	23
3.1.12 LM2596 Buck Converter	23
3.1.13 Push Button.....	24
3.1.14 Slide Button	24
3.2 SOFTWARE DESCRIPTION	25
3.2.1 Arduino Programming Language	25
3.2.2 Arduino IDE	26
3.2.3 Python Programming Language	27
3.2.4 Visual Studio Code	28
3.2.5 PuTTY	29
3.2.6 TigerVNC	30
CHAPTER-4 DESCRIPTION OF THE PROJECT	32-36
4.1 METHODOLOGY.....	32
4.2 CIRCUIT DESIGN	35
4.3 CIRCUIT IMPLEMENTATION	36
CHAPTER-5 RESULT, CONCLUSION & FUTURE SCOPE	37-40
5.1 RESULT	37

5.2 CONCLUSION.....	38
5.3 FUTURE SCOPE	40
REFERENCES.....	41-43
APPENDIX.....	44-71

LIST OF TABLES

Table 1 : Components Used, their Specifications and their function	9
Table 2: Pin Mapping Table of Arduino Nano	11

LIST OF FIGURES

Figure 1: Raspberry Pi 4B Pin Layout	11
Figure 2 : Arduino Nano pinout diagram.....	13
Figure 3 : SIM800L GSM Module with pin layout	15
Figure 4 : Pin Description of NEO6M GPS Module	16
Figure 5 : Joystick Module	18
Figure 6 : USB Mic.....	19
Figure 7 : DC Brushed Motor	20
Figure 9 :L298N Motor Driver Module	20
Figure 10 : MPU6050	22
Figure 11 : Li-ion Cell	23
Figure 12 : 12 to 5 V type A converter	23
Figure 13 : LM2596 Buck Convertor	24
Figure 14 : Push button	24
Figure 15 : Slide Button.....	25
Figure 16 : Arduino IDE Interface	26
Figure 17 : VS Code Interface	29
Figure 18 : Putty Interface to connect with Pi using SSH Client.....	30
Figure 19 : TigerVNC connected to Raspberry Pi.....	31
Figure 20 : Architecture Design of the Wheelchair Prototype	32
Figure 21 : Flow Chart of the Emergency Alert System	34
Figure 22 : Flow Chart of Movement Control	34
Figure 23 : Schematic Diagram for Movement Control	35
Figure 24 : Schematic Diagram for Emergency Features	35
Figure 25 : Circuit Implementation of Movement Control.....	36
Figure 26 : Circuit Implementation of Emergency Control and Location Tracking ...	36
Figure 27 : Final Prototype Image	37
Figure 28 : Screenshots of (a) Help Feature, (b) Fall Alert Feature and (c) Location Tracking	38

CHAPTER-1

INTRODUCTION

1.1 PROJECT OVERVIEW

In many countries, the number of physically challenged and paralyzed people increases every day due to accidents or genetic disorders. Today according to WHO's report, an estimated annual global incidence is 40 to 80 cases per million population [22], which makes them feel worse in certain situations. These people are always dependent on their work. However, they can become self-independent and perform some daily activities on their own with the help of assistive devices. The most widely used assistive devices are "wheelchairs" but there are many disabled people with weak or no energy in limbs and joints due to which they cannot move the wheelchair by themselves.

On reading various articles it is found that the previous work that has been done in this field used Raspberry Pi as the main controller of the system [13], in which the question arises that the Raspberry Pi didn't have GPU due to which it cannot provide the facility to run multiple threads at the same time and this creates delay in the system. Due to this feasibility of those devices is questioned. So, to overcome this here we are using Arduino Nano as the soul controller of our project. Also, in some cases of voice-operated wheelchairs, there is the risk of losing data due to human error because it stores commands by erasing the previously recorded data [9] and for this, we are using Raspberry Pi, separately for voice control commands.

In our project, in case of emergency or help, there is a feature of notifying the concerned authorities with an SOS message with the current location of the wheelchair. The real-time location tracking enables caregivers and users to monitor its position anytime, enhancing safety by enabling quick responses to unexpected situations and for motion independence, the feature of voice control and joystick control is provided. So, for the voice-controlled feature, the input is fed to the Raspberry Pi using the microphone in which speech-to-text conversion is done using the Google APIs which provides better accuracy and less delay when it comes to interacting with human beings.

Then these commands are provided to the Arduino Nano which processes it according to the accessibility of the motor driver unit. Similarly, for joystick-controlled movements, the input is directly given to Arduino Nano via joystick. To monitor the orientation, movements, falls, feedback etc. of the wheelchair a gyroscope sensor and accelerometer are installed whose input is again fed to Nano to check and process the whole commands. For emergencies, help request messages and real-time tracking features, GSM and GPS modules are installed with Nano so that the current live location of the person can be shared with the caregiver.

1.2 OBJECTIVES OF THIS PROJECT

1. **Wheelchair Movement Control using Joystick and Voice Commands:** The primary objective is to provide independence to the disabled people. The joystick feature plays a crucial role, especially to those who are unable to speak and paraplegic. As the joystick moves the wheels will move accordingly to that joystick movement. This technology allows them to control the equipment using hand movements, enabling them to perform tasks and interact with their environment independently. Also, Voice command feature plays a crucial role in improving the quality of life for quadriplegic individuals, who have limited or no use of their limbs. Voice command work for forward, backward, left, right, stop, turn around & help. This innovative feature enables them to control the movement of the wheelchair and thus enhancing their independency and autonomy.
2. **Request Message Feature:** This feature can offer a sense of security and independence to wheelchair users, especially in situations where they may find themselves in need of assistance but are alone, as the independent mobility is provided. For this feature the wheelchair is enabled with the switch, on pressing that switch help message will deliver to the care person. By incorporating a request message feature into the wheelchair's design, individuals can easily communicate their need for help to others, whether it is for simple tasks or for emergencies.
3. **Emergency Alert System:** Emergency alerting wheelchair is equipped with sensors and technology that can detect sudden movements, impacts, or changes in orientation. When an incident such as a fall or collision occurs, these sensors trigger the system to

send an alert to saved contacts. The alert typically includes an alerting message and the current location of the wheelchair user.

4. **Location Tracking:** The feature enables GPS tracking for wheelchairs. When a caregiver or concerned individual sends a request message to share the current location of the wheelchair, the real-time location is then shared with that person. This functionality is particularly useful in scenarios where monitoring the wheelchair's whereabouts is essential for safety and assistance purposes.

1.3 NEED OF THIS PROJECT

As discussed in the introduction, there are several cases where the person is dependent on other people and this type of situation makes them feel worse. Independent mobility is essential for physically disabled people to develop physical, cognitive, communicative, and social skills. The electric wheelchairs are highly priced. Hence, this model aims to build a more complex control scheme for electric wheelchairs. The main problem of the wheelchair is that the disabled person cannot use it, so the type of mechanical assistance that a disabled person needs to move around depends largely on the extent of his or her disability. So, there is no need to manage behind the seat to let another person drive it [2]. After these kinds of complications this project can be used to reduce the human effort of the user and independence by providing self-mobility.

Emergency alerts in wheelchairs are crucial for the safety and well-being of physically handicapped individuals. These alerts serve various purposes, including notifying caregivers or emergency services about the user's condition. Wheelchair users may encounter unforeseen situations like accidents or medical emergencies. An emergency alert system can instantly notify caregivers or authorities for prompt assistance. Having an emergency alert system empowers wheelchair users to have greater independence, knowing that help is readily available in emergencies.

So, the above-mentioned needs are the wholesome problem which we are focusing on here. The locomotive assistance and the emergency assistance is hence provided by the features of voice control, joystick control and emergency alert.

CHAPTER-2

LITERATURE REVIEW

Agarwal et al. (2016) employed a voice-operated wheelchair designed to assist individuals unable to move their limbs independently, including those with Parkinson's disease. The study introduced an Elechouse voice recognition module V-3 with a wireless transmitter and receiver section, replacing the joystick, as per the World Health Organization's report indicating a low count of speech impediments. This technology allows users to operate the wheelchair through voice commands, supplemented by a voice authentication process.

Meena et al. (2017) introduced an automated wheelchair system utilizing voice recognition technology. That voice-controlled wheelchair simplifies mobility for individuals with physical disabilities who lack hand control. It operates on motor power for movement and responds to voice commands. The system consists of an Arduino, HM2007 voice recognition module, and motors. Upon receiving a command, the voice recognition module identifies it and sends the corresponding coded data from memory to the Arduino Microcontroller, which adjusts the wheelchair's movement accordingly. Additionally, the wheelchair includes a joystick option for those physically disabled individuals capable of hand movement.

Sukerkar et al. (2018) proposed that in today's society, numerous individuals with disabilities encounter challenges in performing daily movements and activities, often relying on assistance from others. However, through the utilization of assistive devices, such individuals can achieve greater independence and perform tasks autonomously. Among these devices, wheelchairs are prevalent, offering mobility to those unable to walk due to several reasons such as illness, disability, or injury. Yet, many individuals with limited limb and joint function struggle to maneuver traditional wheelchairs. Hence, the introduction of smart wheelchairs presents a promising solution for enhancing accessibility and independence for both users and society as a whole. Smart wheelchairs, equipped with electric power, incorporate additional components such as computers and sensors, facilitating easier and more efficient handling by users or their caregivers. The advancement of Artificial Intelligence, sensor

technology, and robotics has spurred the evolution of wheelchairs, introducing new features and capabilities. This paper provides an overview of the current state of smart wheelchairs and outlines avenues for future research in this domain.

Sangeetha et al. (2019) outlined the development and execution of an IoT-based autonomous wheelchair operated via voice commands. This innovation allows individuals with physical disabilities to navigate various locations simply by speaking commands. Moreover, in case of emergencies, the system can automatically place a call to the user's designated guardian. Additionally, the wheelchair includes an alarm system to remind the user to take medication at specified intervals. Notably, the wheelchair responds exclusively to the voice commands of its designated user, enhancing accessibility and control. The mobile phone served as the interface for issuing voice commands, which were then transmitted to the IoT system. Subsequently, the motor adjusted its rotation based on the input received from the IoT.

Renuka et al. (2020) devised a Smart Wheelchair operated through user commands to enable autonomous mobility, reducing the user's physical exertion and fostering wheelchair utilization. This innovation offers physically impaired individuals the opportunity for independent movement between various locations. The primary focus of their study was to gather information and construct a wheelchair utilizing Raspberry Pi and an RF controller to address challenges faced by individuals with physical disabilities. The wheelchair is equipped with an RF module, GSM and GPS modules, servo motor, and is controlled as a whole unit. Users provide input commands to guide the wheelchair's movement, interfacing through RF communication and voice commands, depending on user preference. An emergency solution was proposed, incorporating an alert system. Movements and hand gestures from visually impaired individuals are recognized and compared with stored models on the receiver. If a match is found, home appliances are managed accordingly.

Hameed et al. (2021) highlights that declining mobility significantly impacts the independent living capabilities of elderly and disabled individuals. To enhance living conditions, mobility assistive technologies are being developed, but there's room for improvement in current devices. This study delves into the design and construction of a smart wheelchair featuring multiple control interfaces. A prototype of the smart wheelchair was constructed based on an existing traditional wheelchair model,

integrating similar electrical and mechanical enhancements. To enhance user engagement, the system incorporates speech and gesture control interfaces, alongside a mobile application for wheelchair control. Detailed information is provided regarding the mechanical design and electrical and control systems of the smart wheelchair. Experimental validation of the developed smart wheelchair was conducted, along with a user study to evaluate preferences among the various control interfaces, with the results presented.

Rahimunnisa et al. (2021) investigated automated wheelchairs equipped with sensors and a data processing unit, constituting a specialized category of wheeled mobile robots commonly referred to as "smart wheelchairs." In the current system, wheelchair movement controlled by a joystick employs buttons for starting and stopping the wheels, which can be challenging for individuals with disabilities to operate precisely. Although there are smart wheelchairs with gesture control, they often lack accuracy in location calculation. The proposed system utilizes artificial intelligence, presenting a unique integration of a wheelchair and health monitoring system. These wheelchairs offer both manual and automatic modes of access. In manual mode, the joystick controls the wheel, while in automated mode, the MPU6050 sensor and accelerometer control direction through gestures. An SPO2 sensor integrated into the wheelchair collects health parameters, promoting user self-dependency. Additionally, deep learning analysis of sensor data and wheelchair usage patterns is conducted. The stress level is assessed by comparing it with the dataset. Sensor signals are continuously monitored, and vital data is regularly updated on the ThingSpeak website through a Bluetooth module, functioning as a digital health chart.

Wada et al. (2021) introduces a joystick car drive system tailored for individuals with disabilities. With this system, we can operate a car using just one hand while seated in an electric wheelchair. The joystick allows us to control acceleration or deceleration by moving it forward or backward and steering by moving it left or right. This means that even if one has limited arm mobility or strength, as well as leg disabilities, one can independently drive a car. Furthermore, there is no need to transfer from wheelchair to a car seat, as the van is equipped with a lift at the back, allowing us to easily access the driver's position by propelling wheelchair inside. To ensure driving safety in case of system failure, gas and brake pedals can be manually operated via a mechanical linkage

connected to the joystick lever. The steering wheel is turned by an electric motor controlled by a micro-computer system, which maintains a precise steering angle based on joystick input. To adapt the system for wheelchair seating, a steering drive mechanism incorporating a DC motor, magnetic clutch, potentiometer for angle detection, and transmission gears was designed. The prototype mechanism was installed in a real van's cockpit and tested to verify the feasibility of the proposed joystick-controlled car-wheelchair mobile system.

Thakare et al. (2022) introduced an advanced system that offers convenience for individuals with disabilities, particularly those with paralysis or leg disabilities who struggle to move around independently. Many resort to using a walker, but this can lead to loss of balance and injuries. Their system addresses these challenges by providing a powered wheelchair solution powered by Raspberry Pi. It includes a Bluetooth module, GSM and GPS module, driver motor, and wheelchair. Users can command the wheelchair's movement via input commands. The transmitter and receiver circuit communicate via RF through the Bluetooth module.

Cui et al. (2023) conducted research on intelligent wheelchairs, primarily focusing on motion control, while the investigation into attitude-based adjustments is relatively lacking. This article proposes a method for adjusting the posture of intelligent wheelchairs based on action intention recognition. The method explores the correlation between force changes on the contact surface between the human body and the wheelchair and the intended actions. It is implemented on a multi-part adjustable electric wheelchair equipped with multiple force sensors to gather pressure data from different areas of the passenger's body. At the system's upper level, pressure data is converted into a pressure distribution map, shape features are extracted using the VIT deep learning model, and actions are identified and classified to determine passengers' intentions. The electric actuator adjusts the wheelchair's posture according to these intentions. Through testing, this method effectively collects passengers' body pressure data with over 95% accuracy in recognizing common intentions such as lying down, sitting up, and standing up. The wheelchair can then adapt its posture, accordingly eliminating the need for additional equipment and reducing the impact of external factors on users.

Thongpance et al. (2023) conducted research to develop an electric wheelchair equipped with mecanum wheels, enabling omnidirectional movement controlled by a joystick. This innovation represents a notable advancement over traditional electric wheelchairs, which can only move linearly. Their design incorporates a joystick controller and utilizes an average filter to enhance joystick processing. Adhering to ISO 2570-2555 guidelines, they employed Arduino DUE as the primary processor for wheel rotation control. By determining speed gain and angle using resultant vector analysis of the analog joystick, they achieved precise control over wheel direction and speed. The resulting electric wheelchair, conforming to standard structure, successfully moves in the intended direction and speed according to joystick input, demonstrating the effectiveness of their design and construction approach. The authors emphasize the significance of joystick control development for electric wheelchairs, facilitating the creation of innovative designs such as the one presented in their study.

CHAPTER-3

COMPONENT DESCRIPTION

3.1 HARDWARE DESCRIPTION

Table 1 : Components Used, their Specifications and their function

S.No.	Components	Specifications	Function
1	Raspberry Pi	4B with 4GB RAM	Voice Command Recognition
2	Arduino Nano	AtMega328p	Main Controlling
3	GSM	800l	For SMS
4	GPS	Neo 6m, 5Hz	Location Detection
5	Joystick Module	Analog	Use to give input for movements.
6	Microphone	USB	To give voice command inputs
7	DC Motor	Brushed, 60RPM	Movement
8	Motor Driver	L298N	Driving Motors
9	MPU-6050	3-axis Gyroscope & Accelerometer	Wheelchair State Recognition
10	Batteries	Li-ion, 3.6V, 2600mAH	To Power system
11	Buck Converter	LM2596	To regulate voltage for GSM
12	Buck Converter	Type A	To power Raspberry Pi
13	Push Button	-	For SOS
14	Slide switch	SPDT	Switch b/w Voice & Joystick

3.1.1 Raspberry Pi 4B

The Raspberry Pi 4 Model B represents a significant advancement in single-board computing, boasting a powerful quad-core ARM Cortex-A72 processor running at up to 1.5GHz. With options for 2GB, 4GB, or 8GB of LPDDR4 RAM, it offers improved performance and multitasking capabilities over its predecessors. The inclusion of dual micro-HDMI ports supports dual 4K display output, while Gigabit

Ethernet and dual-band 802.11ac wireless networking ensure seamless connectivity. USB 3.0 ports enable faster data transfer rates and expanded peripheral compatibility. Its GPIO header and robust community support make it ideal for DIY electronics projects and educational endeavours. Whether used for learning programming, powering home automation systems, or serving as the brains behind innovative IoT projects, the Raspberry Pi 4 Model B continues to inspire creativity and innovation across various fields. Its compact size, affordability, and versatility make it an indispensable tool for enthusiasts, educators, and professionals alike.

3.1.1.1 Raspberry Pi Specifications:

- Processor: Quad-core ARM Cortex-A72 CPU running at up to 1.5GHz
- RAM: Options for 2GB, 4GB, or 8GB LPDDR4 RAM
- Connectivity:
 - Gigabit Ethernet
 - Dual-band 802.11ac wireless networking
 - Bluetooth 5.0
- Video Output: Dual micro-HDMI ports supporting up to 4K resolution
- USB Ports:
 - 2 × USB 3.0 ports
 - 2 × USB 2.0 ports
- GPIO: 40-pin GPIO header
- Storage: MicroSD card slot for operating system and data storage
- Power: USB-C power connector
- Dimensions: 85.6mm × 56.5mm × 17mm

3.1.1.2 Raspberry Pi Features:

- Enhanced Processing Power: Provides improved performance over previous models, suitable for a wide range of tasks.
- Expanded RAM Options
- Comprehensive Connectivity: Includes Gigabit Ethernet, dual-band Wi-Fi, and Bluetooth, ensuring seamless networking and communication.
- High-resolution Video Output: Supports dual 4K displays via micro-HDMI ports, enabling versatile display setups.
- Compact Form Factor

- **Operating System Compatibility:** Supports various operating systems, including Linux distributions and Windows 10 IoT Core.
- **Versatile Accessories:** Compatible with a wide range of accessories, enhancing its functionality and adaptability for different use cases.

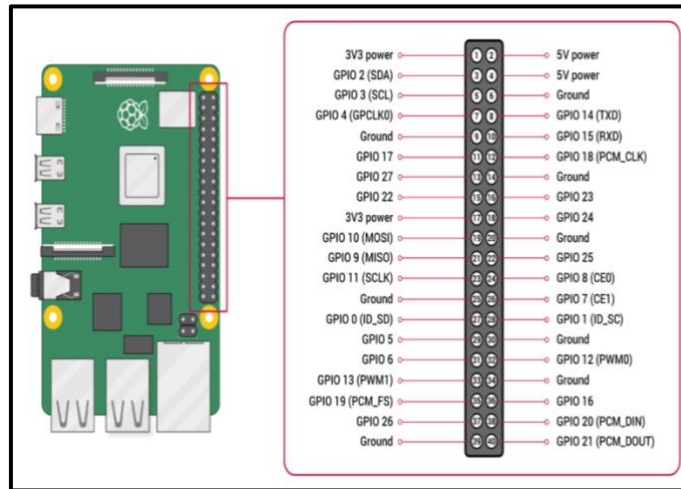


Figure 1: Raspberry Pi 4B Pin Layout

3.1.2 Arduino Nano

The Arduino Nano is a compact yet powerful microcontroller board that is widely used in electronics projects and prototyping. Developed by Arduino, the Nano is part of the Arduino ecosystem, known for its simplicity, versatility, and ease of use. With its small form factor and robust capabilities, the Arduino Nano has become a popular choice among hobbyists, students, and professionals alike for a wide range of applications.

3.1.2.1 Arduino Nano Specification and Features

Table 2: Pin Mapping Table of Arduino Nano

Pin Number	Pin Name	Function
1	D0 (RX)	Digital I/O, UART RX
2	D1 (TX)	Digital I/O, UART TX
3	D2	Digital I/O
4	D3	Digital I/O, PWM
5	D4	Digital I/O
6	D5	Digital I/O, PWM

7	D6	Digital I/O, PWM
8	D7	Digital I/O
9	D8	Digital I/O
10	D9	Digital I/O, PWM
11	D10	Digital I/O, PWM, SPI SS
12	D11	Digital I/O, PWM, SPI MOSI
13	D12	Digital I/O, SPI MISO
14	D13	Digital I/O, SPI SCK, Built-in LED
15	A0	Analog Input
16	A1	Analog Input
17	A2	Analog Input
18	A3	Analog Input
19	A4 (SDA)	Analog Input, I2C SDA
20	A5 (SCL)	Analog Input, I2C SCL
21	A6	Analog Input
22	A7	Analog Input
23	VIN	External power input (7-12V)
24	GND	Ground
25	RST	Reset
26	5V	Regulated 5V
27	3.3V	3.3V from FTDI chip
28	GND	Ground
29	AREF	Reference voltage for analog inputs
30	IOREF	Voltage reference for shields

- General Specifications
 - Microcontroller: ATmega328P (Arduino Nano V3.x) or ATmega168 (older versions)
 - Operating Voltage: 5V
 - Input Voltage (recommended): 7-12V
 - Input Voltage (limits): 6-20V

- Digital I/O Pins: 14 (6 provide PWM output)
- Analog Input Pins: 8
- DC Current per I/O Pin: 40 mA
- Flash Memory: 32 KB (ATmega328P) / 16 KB (ATmega168), with 2 KB used by the bootloader
- SRAM: 2 KB (ATmega328P) / 1 KB (ATmega168)
- EEPROM: 1 KB (ATmega328P) / 512 bytes (ATmega168)
- Clock Speed: 16 MHz

Following is the pin mapping of Arduino Nano:

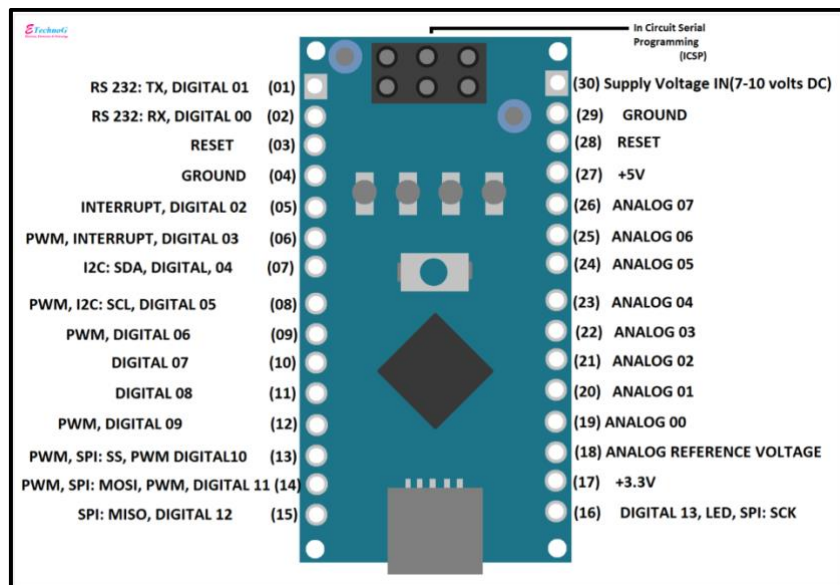


Figure 2 : Arduino Nano pinout diagram

- Key Features
 - Compact Size:
 - Ideal for projects with space constraints.
 - Breadboard-friendly design.
 - Microcontroller:
 - ATmega328P or ATmega168, providing reliable performance for various applications.
 - 16 MHz clock speed for efficient processing.
- Memory:
- Adequate memory for small to medium-sized projects.
 - 32 KB Flash memory, 2 KB SRAM, and 1 KB EEPROM on ATmega328P.

- Built-in Features:
 - Reset Button: Convenient for resetting the board during development.
 - Built-in LEDs: TX and RX LEDs for serial communication indication and an additional LED on pin D13 for simple status indications.
 - Bootloader: Pre-installed bootloader for easy code uploading without the need for an external programmer.

3.1.3 SIM800L GSM Module

The SIM800L is a GSM/GPRS module that can be used to provide a GSM network connection for sending and receiving voice calls, SMS, and data. It is widely used in embedded systems and IoT projects due to its small size, low power consumption, and extensive features.

3.1.3.1 Features of GSM Module

- Quad-Band GSM:
 - Supports GSM/GPRS 850/900/1800/1900MHz frequencies.
 - This allows it to operate in virtually any part of the world.
- Communication Capabilities:
 - Voice Calls: Make and receive voice calls.
 - SMS: Send and receive SMS messages.
 - GPRS: Provides internet connectivity via GPRS, enabling data transmission.
- Size and Power:
 - Compact design, making it suitable for space-constrained applications.
 - Operates on a supply voltage range of 3.4V to 4.4V (typically 4.0V).
 - Low power consumption, ideal for battery-powered applications.
- Interfaces:
 - UART Interface: Communicates with microcontrollers using UART (3.3V logic level).
 - AT Commands: Uses standard AT commands for controlling GSM functions.
- Additional Features:
 - Bluetooth: Some versions support Bluetooth for short-range wireless communication.

- RTC: Real-Time Clock for timekeeping.
- PWM: Pulse Width Modulation for controlling peripherals like LEDs and motors



Figure 3 : SIM800L GSM Module with pin layout

3.1.4 NEO 6M GPS Module

The NEO-6M GPS module is a popular GPS receiver module used in various applications for tracking and navigation. It is known for its accuracy, reliability, and ease of integration into microcontroller-based projects.

3.1.4.1 Features of NEO 6M GPS Module

- High Sensitivity:
 - High-performance receiver with -161 dBm tracking sensitivity.
 - Fast time to first fix (TTFF) and strong signal acquisition.
- Compact Size, Small form factor, making it suitable for space-constrained applications.
- Power Efficiency: Low power consumption, typically around 45 mA during operation.
- Interfaces: UART interface for easy communication with microcontrollers.
- Data Output: Provides GPS data and time information in NMEA format.
- Built-in EEPROM stores configuration settings and almanac data for quicker start-up times.
- Integrated Antenna:
 - Includes a ceramic patch antenna for improved signal reception.
 - Optional external antenna support for enhanced performance.

3.1.4.2 Technical Specifications of NEO 6M GPS Module

- Receiver Type: 50 channels, GPS L1 frequency (1575.42 MHz)
- Sensitivity:
 - Acquisition: -148 dBm
 - Tracking: -161 dBm
- Update Rate: 1 Hz (default), up to 5 Hz
- Position Accuracy: 2.5 meters CEP (Circular Error Probable)
- Time to First Fix (TTFF):
 - Cold start: 27 seconds
 - Warm start: 25 seconds
 - Hot start: 1 second
- Supply Voltage: 3.3V to 5V
- Operating Temperature: -40°C to +85°C

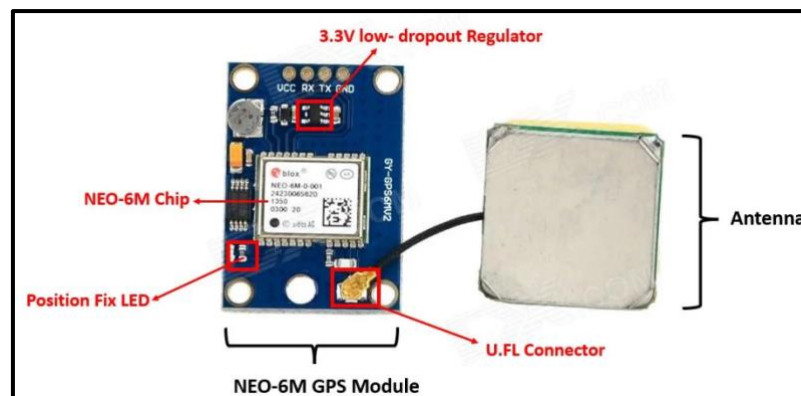


Figure 4 : Pin Description of NEO6M GPS Module

3.1.5 Joystick Module

A joystick module typically consists of two potentiometers and a pushbutton, allowing it to provide analog input for X and Y axes and a digital input for button presses. Here are the detailed specifications of a common joystick module used in hobbyist and educational projects: The voltage given to joystick must be 5V (preferably) and remain constant during calibration and operation to avoid any drift.

3.1.5.1 General Specifications of joystick module

- Operating Voltage: 3.3V to 5V, compatible with most microcontrollers.
- Operating Current: Typically, around 10 mA.

- Dimensions: Approximately 34mm x 26mm x 32mm (L x W x H).

3.1.5.2 Electrical Specifications of joystick module

- X and Y Axes (VRx and VRy):
 - Type: Analog potentiometers.
 - Output Voltage Range:
 - 0V to Vcc (where Vcc is typically 5V when powered with 5V).
 - Resolution: Depends on the ADC resolution of the microcontroller (e.g., 10-bit ADC provides 1024 discrete levels).
- Pushbutton (SW):
 - Type: Digital switch.
 - Output State:
 - Normally open (NO)
 - Internal Pull-Up

3.1.5.3 Mechanical Specifications of joystick module

- Joystick Movement:
 - Range: Full 360-degree movement.
 - Return Mechanism: Spring-loaded to return to the center position when released.
 - Travel Distance: Typically, around ± 10 degrees in each direction from the center position.
- Button Actuation:
 - Actuation Force: Varies, typically around 200-300 grams of force.
 - Travel Distance: Around 1.5mm to 2mm.
- Typical Output Characteristics
 - Analog Output (VRx and VRy)
 - Center position (no movement): Approx. half of Vcc (e.g., ~2.5V if Vcc is 5V).
 - Full left/right (VRx) or up/down (VRy): 0V to Vcc (e.g., 0V to 5V).
 - Proportional to the joystick's position, allowing for precise directional control.
 - Digital Output (SW):

- Unpressed: High (Vcc, e.g., 5V),
- Pressed: Low (0V).
- Environmental Specifications
 - Operating Temperature: -10°C to +70°C.
 - Storage Temperature: -20°C to +80°C.

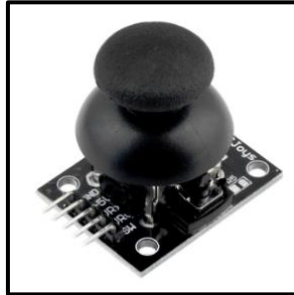


Figure 5 : Joystick Module

3.1.6 USB Microphone

Raspberry Pi USB Plug and Play Desktop Microphone is USB Microphone that offers compatibility with any plug-and-play enabled Raspberry Pi Model 5, 4, B+, 2 model B, Raspberry Pi 3 as wells as it is also compatible with PC and Mac.

3.1.6.1 Features of USB Microphone:

- Made of plastic, light and durable.
- Simple to operate as a single USB plug-&-play connection.
- Noise-cancelling microphone filters out unwanted background noise..
- Microphone pivots on base to hold the preferred position.
- Flexible neck to adjust your convenient direction and Height for talking.

3.1.6.2 Specification of USB Microphone

- Good Capture Range (m) = 1
- Sensitivity = -67 dBV / pBar, -47 dBV / Pascal +/- 4dB
- Reduction of the sensitivity = -3dB at 1.5V
- Frequency response (kHz) = 100 ~ 16
- Material = Plastic and metal
- Cable Length = 750mm
- Weight = 0.04 kg

- Dimensions = Base : 53mm
- Total Length : 135mm
- Mic head: 50mm X 17mm



Figure 6 : USB Mic

3.1.7 DC Motor

12V, 60 RPM DC brushed motor is a specific type of electric motor designed to operate at 12 volts of direct current (DC) and achieve a rotational speed of 60 revolutions per minute (RPM). The L298N H-bridge module with on-board voltage regulator motor driver can be used with this motor that has a voltage of between 5 and 35V DC.

3.1.7.1 Specification of DC Brushed Motor

- Voltage Rating:
- Speed: 60 RPM (under no-load conditions)
- Type: Brushed Motor
- Torque: 2kg-cm
- Current:
 - No-load Current: 60mA
 - Load Current: 300mA
- Metal Gears



Figure 7 : DC Brushed Motor

3.1.8 L298N Motor Driver Module

The L298N motor driver module is a popular choice for controlling DC motors and stepper motors in various robotics and automation projects.

3.1.8.1 Features:

- Dual H-Bridge Design : Module can control two DC motors independently
- Operating Voltage:
 - Logic Voltage (Vss): 5V
 - Motor Supply Voltage (Vs): 5V to 35V
- Current Handling:
 - Continuous Current: Up to 2A per channel.
 - Peak Current: Up to 3A per channel.
- PWM Control:
 - Supports Pulse Width Modulation (PWM) for speed control of the motors.

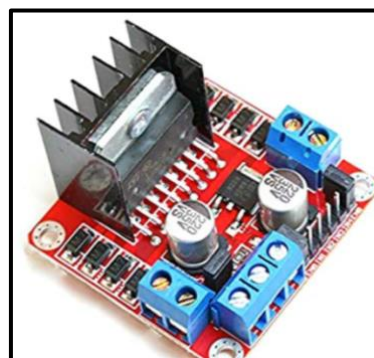


Figure 8 :L298N Motor Driver Module

- Thermal and Short Circuit Protection
- Control Pins:

- Enable Pins: ENA and ENB to enable or disable the motor channels.
 - Input Pins: IN1, IN2, IN3, and IN4 to control the direction of the motors.
- Power Indicators: On-board LEDs to indicate power and motor activity

3.1.9 MPU6050

The MPU6050 is a popular and versatile sensor module that combines a 3-axis gyroscope and a 3-axis accelerometer in a single chip, making it a favorite among hobbyists and professionals for motion tracking and orientation sensing applications. The MPU6050 is a powerful and versatile sensor, ideal for a wide range of applications requiring motion detection and orientation sensing. Its integration of a gyroscope and accelerometer, along with additional features like the DMP, make it a comprehensive solution for many projects.

3.1.9.1 Features:

- 6 Degrees of Freedom (6-DoF):
 - 3-Axis Gyroscope: Measures rotational velocity in degrees per second (dps).
 - 3-Axis Accelerometer: Measures linear acceleration in meters per second squared (m/s^2).
- Digital Motion Processor (DMP):
 - An integrated DMP that can process complex motion algorithms directly on the sensor, offloading this task from the host microcontroller.
- I2C (primary) and SPI (secondary) Communication:
- Built-in Temperature Sensor
- Programmable Low-Pass Filters that helps to reduce noise in the sensor data.
- FIFO Buffer to store sensor data, which can be useful for reducing the burden on the host processor.

3.1.9.2 Specifications:

- Supply Voltage: 2.3V to 3.4V
- Gyroscope Range: ± 250 , ± 500 , ± 1000 , ± 2000 dps
- Accelerometer Range: $\pm 2g$, $\pm 4g$, $\pm 8g$, $\pm 16g$
- Communication Protocol: I2C (up to 400kHz) and SPI (up to 1MHz)
- Operating Temperature Range: -40°C to $+85^{\circ}\text{C}$.

- Weight: 2.1g



Figure 9 : MPU6050

3.1.10 Battery

A 3.6V 2600mAh Li-ion battery is a common rechargeable battery used in various electronic devices due to its high energy density, reliability, and long life cycle. 18650 Li-ion battery, which is a rechargeable battery, can be used in many different Industrial, Automation and Military applications, laptops, powerbanks cameras and battery packs.

3.1.10.1 Features:

- Voltage: 3.6 V (nominal)
- Capacity: 2600mAh
- Life Cycle: Typically, Li-ion batteries can endure hundreds of charge/discharge cycles before their capacity significantly degrades. Usually the life of 2600mAh cells is 3000 cycles, it is recommended to be replaced after 3000 cycles.
- Charge and Discharge Characteristics:
 - Charge Voltage: 4.2V (maximum)
 - Discharge Cut-off Voltage: Usually around 2.75V to 3.0V
 - Standard Charge Current: Often around 0.52A
 - Standard Discharge Current: 1.3A
 - Max Pulse Discharge Current: 2.6A
- Dimensions: 8mm in diameter and 65mm in length
- Weight: Typically around 45 grams.



Figure 10 : Li-ion Cell

3.1.11 12V to 5V Type A Buck Converter

This is a DC to DC 6-24V to 5V USB Output Step Down Power Charger Buck Converter. On-board female type A USB port to power any electronic gadget which requires 5V input power up to 3A

3.1.11.1 Specifications:

- Maximum Conversion Efficiency: 97.5%
- Full load temperature: 30 A/°C
- Switching Frequency: 500 kHz
- Load Regulation: $\pm 1\%$
- Voltage Regulation: $\pm 0.5\%$
- Dimensions: 26.4 mm x 15 mm x 7.4mm (L x W x H)
- Weight: 9g

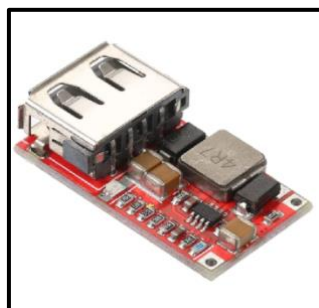


Figure 11 : 12 to 5 V type A converter

3.1.12 LM2596 Buck Converter

The LM2596 regulator is monolithic integrated circuit ideally suited for easy and convenient design of a step-down switching regulator (buck converter).

3.1.12.1 Specifications:

- Output current: 2A(rated), 3A (max.)(Additional heat sink required)
- Switching Frequency: 150KHz
- Operating temperature: Industrial grade (-40°C to +85°)
- Conversion efficiency: 92%(highest)
- Load Regulation: ± 0.5
- Dynamic Response Speed: 5% 200uS
- Dimension: 45mm x 20mm x 14mm (L x W x H)

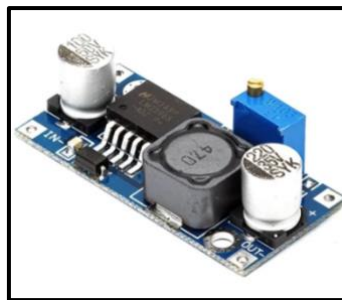


Figure 12 : LM2596 Buck Convertor

3.1.13 Push Button

A push-button (also spelled pushbutton) or simply button is a simple switch mechanism to control some aspect of a machine or a process. It is used to send SOS signal.



Figure 13 : Push button

3.1.14 Slide Button

The slide button is used for switching between voice and joystick control.

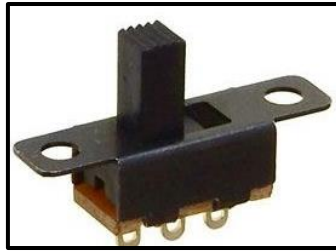


Figure 14 : Slide Button

3.2 SOFTWARE DESCRIPTION

3.2.1 Arduino Programming Language

The Arduino Programming Language is based on the Wiring development platform, which, in turn, is built upon Processing. This lineage ensures compatibility and familiarity for developers. While the syntax resembles C++, there are some simplifications and additional features. This makes it accessible even to beginners who might find full C++ daunting. The Arduino ecosystem includes libraries that provide pre-built functions and components. These libraries simplify tasks like interfacing with sensors, controlling servos, or handling communication protocols.

3.2.1.1 Features

- **Cross-Platform:** Arduino is accessible across different platforms, including Windows. This flexibility allows developers to work seamlessly regardless of their operating system.
- **Extensible Software:** Experienced programmers can enhance the Arduino language by importing C/C++ frameworks and libraries. This extensibility enables customizations and tailored solutions.
- **Open Source:** Both the Arduino software and hardware are open source. This means users have access to the source code and can modify it to suit their specific needs.
- **Based on C++:** The Arduino language is based on C++, a widely used and well-known programming language.

3.2.2 Arduino IDE

The Arduino Integrated Development Environment (IDE) is the software used to write and upload code to the Arduino board. It provides a user-friendly interface for creating and managing Arduino programs also called sketches. The IDE is compatible with various Arduino boards, which allows you to choose the right one according to the requirement.

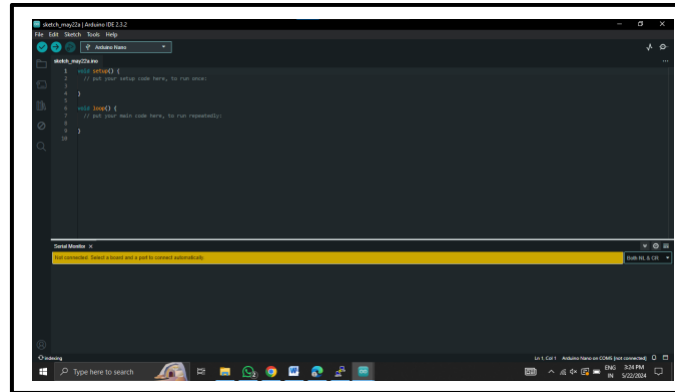


Figure 15 : Arduino IDE Interface

3.2.2.1 Features

- **User-Friendly Interface:** The IDE provides an intuitive interface for creating and managing Arduino sketches (programs).
- **Board Compatibility:** The IDE supports various Arduino boards, allowing you to select the appropriate one based on your project requirements. Whether you're using an Uno, Nano, or Mega, the IDE has you covered.
- **Code Compilation:** When you create a sketch, the IDE processes and compiles it into machine language that the Arduino board can execute. This compilation step ensures that your code is ready for execution.
- **Example Sketches:** The IDE comes with a collection of example sketches that demonstrate various functionalities. These serve as great starting points for beginners and help you understand how to use different sensors, actuators, and communication modules.
- **Serial Monitor:** The IDE includes a Serial Monitor feature, allowing you to communicate with the Arduino board via the USB connection. You can send and receive data, debug your code, and monitor sensor readings.

- **Library Manager:** The IDE has a built-in Library Manager that simplifies the process of adding external libraries. You can easily search for and install libraries to extend the capabilities of your sketches.

3.2.3 Python Programming Language

Python is a versatile language used in various domains, from web development to data science. Its readability and clean syntax make it accessible to beginners and experts alike. Python is the officially recommended language for Raspberry Pi programming. The Raspberry Pi Foundation provides Python libraries and tools that simplify hardware interaction. Python's straightforward syntax allows one to focus on solving problems rather than dealing with complex language constructs. Python boasts a rich ecosystem of libraries. For Raspberry Pi, these libraries cover everything from GPIO control to handling sensors, displays, and communication protocols. The Raspberry Pi has General Purpose Input/Output (GPIO) pins that allows one to connect external components directly to the board. Python lets you control these pins, read sensor inputs, and send output signals.

3.2.3.1 Features

- **Free and Open Source:** Python is freely available on the official website. Being open source, its source code is accessible to the public, allowing users to download, use, and share it.
- **Easy to Code:** Python's high-level nature makes it easy to learn and write code. Whether you're a beginner or an experienced developer, Python's simplicity and developer-friendly syntax allow you to create programs quickly.
- **Easy to Read:** Python's straightforward syntax relies on indentation rather than semicolons or brackets. This readability makes understanding and maintaining code a breeze.
- **Object-Oriented Language:** Python fully supports object-oriented programming. Concepts like classes, object encapsulation, and inheritance are integral to Python.
- **GUI Programming Support:** You can create graphical user interfaces (GUIs) using modules like PyQt5, wxPython, or Tkinter. PyQt5 is popular for building graphical apps with Python.

- **High-Level Language:** Python abstracts system architecture and memory management, allowing developers to focus on solving problems without low-level details.
- **Large Community Support:** Python's popularity has led to an enormous community on platforms like Stack Overflow. You'll find answers to many Python-related questions there.
- **Easy to Debug:** Python provides excellent error tracing information. Understanding Python's error messages helps you quickly identify and correct issues in your code.
- **Portability:** Python is a portable language. Code written for one platform (e.g., Windows) can run on others (Linux, Unix, Mac) without modification.
- **Integration:** Python can easily integrate with other languages like C and C++.

3.2.4 Visual Studio Code

Visual Studio code or VS code is source code editor which was developed by Microsoft. VS Code runs on Windows, macOS, and Linux. You can use it seamlessly across different operating systems.

3.2.4.1 Features

- **Cross-Platform:** VS Code runs on Windows, macOS, and Linux, making it versatile and accessible across different operating systems. Whether you're on a Windows PC, a Mac, or a Linux machine, you can use VS Code seamlessly.
- **Lightweight and Fast:** VS Code is designed to be lightweight, responsive, and efficient. It doesn't consume excessive system resources, allowing you to focus on coding without slowdowns.
- **Extensions and Customization:** VS Code supports a wide range of extensions. You can enhance its functionality by installing extensions for specific languages, themes, debugging tools, and more. The marketplace offers thousands of extensions to choose from.
- **Integrated Terminal:** VS Code includes an integrated terminal, allowing you to run commands directly within the editor. This streamlines tasks like compiling code, running scripts, and managing Git repositories.
- **IntelliSense and Autocompletion:** This provides IntelliSense, allowing context-aware autocompletion suggestions as you type thus reducing the manual effort.

- **Git Integration:** Git is seamlessly integrated into VS Code. You can manage version control, commit changes, and collaborate with others using Git features directly within the editor.
- **Debugging Tools:** VS Code offers robust debugging capabilities for various programming languages. You can set breakpoints, inspect variables, and step through code to identify and fix issues.

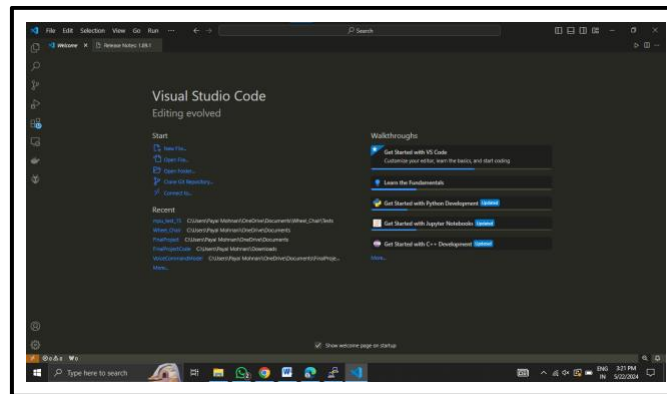


Figure 16 : VS Code Interface

- **Raspberry Pi Programming:** When working with Raspberry Pi, you can use VS Code to write, edit, and debug Python, C++, or other languages. It integrates well with the Raspberry Pi OS (formerly Raspbian).

3.2.5 PuTTY

PuTTY is an SSH client that allows you to connect remotely to your Raspberry Pi over a network. It provides a secure way to access the command-line interface (CLI) of your Raspberry Pi. PuTTY can be used to establish a secure connection to Pi, without having a dedicated physical monitor connected to it, using just your main laptop or PC.

3.2.5.1 Features

- **SSH Client:** PuTTY is an SSH client that allows you to connect remotely to other computers.
- **Secure Communication:** It ensures secure communication by encrypting data during transmission.
- **Cross-Platform:** PuTTY runs on Windows, macOS, and Linux, making it versatile.
- **Tunneling:** You can use PuTTY to tunnel other software (like VNC) over SSH connections.

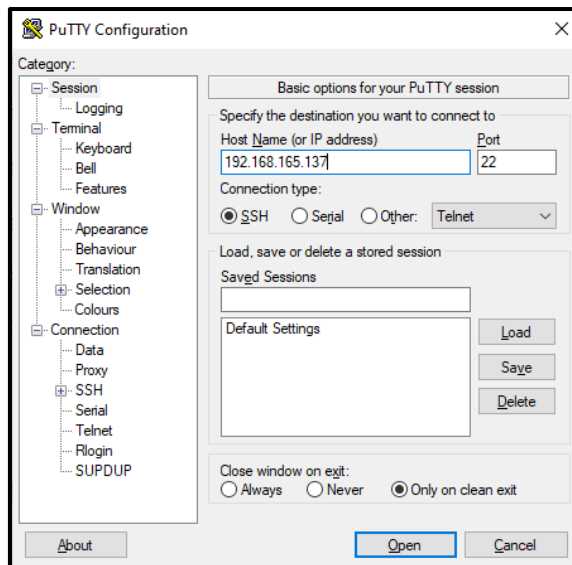


Figure 17 : Putty Interface to connect with Pi using SSH Client

3.2.5.2 Installation of PuTTY

- Download PuTTY Installer from the official PuTTY website and save it.
- Execute the Installer.
- Follow the on-screen instructions to proceed with the installation.
- Stick with the default settings unless you have specific preferences.
- Once the installation is complete, PuTTY, PSFTP, and PuTTYgen will be added to your Start menu. You're all set to use PuTTY for SSH and telnet connections!

3.2.6 TigerVNC

It is a Virtual Network Computing (VNC) server for Raspberry Pi which allows you to remotely access the desktop interface (GUI) of your Raspberry Pi from your laptop.

3.2.6.1 Features

- Graphical Desktop Sharing: TigerVNC enables graphical remote control of other computers.
- Client-Server Model: A server (vncserver) shares its output, and a client (vncviewer) connects to it.
- Performance: TigerVNC provides good performance for 3D and video applications.
- Extensions: It supports advanced authentication methods and TLS encryption.

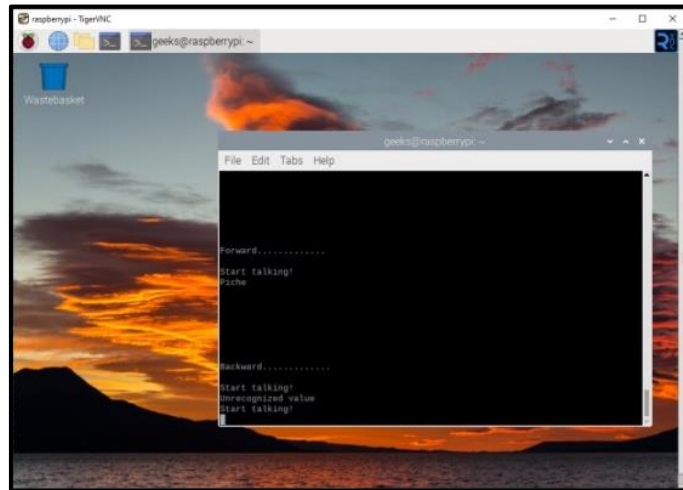


Figure 18 : TigerVNC connected to Raspberry Pi

3.2.6.2 Installation and Configuration:

- Install TigerVNC on your Raspberry Pi.
- Configure it to serve as a VNC server, allowing graphical access.
- You can then use a VNC viewer (such as TigerVNC viewer) on your laptop to connect to the Pi's desktop.

CHAPTER-4

DESCRIPTION OF THE PROJECT

4.1 METHODOLOGY

This project presents a comprehensive solution to enhance the functionality and safety of a wheelchair. The system offers dual control methods - joystick and voice inputs - alongside emergency alerting features such as an SOS button and GPS tracking. The system architecture comprises five layers, namely User Command, Command Processing, Main Control, Communication, and Sensing, and the driver or application layer.

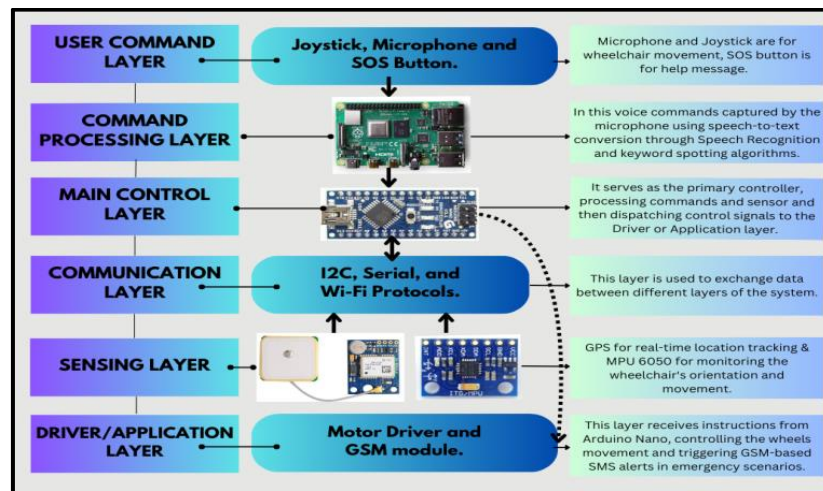


Figure 19 : Architecture Design of the Wheelchair Prototype

This project details a sophisticated wheelchair system designed to significantly enhance independent mobility and safety for individuals with various physical disabilities. The system architecture is composed of multiple integrated layers, each performing specific functions to ensure seamless operation.

The User Command layer is the primary interface for user inputs, accepting signals from a joystick, a microphone for voice commands, and an SOS button for emergency situations. At the heart of the Command Processing layer is the Raspberry Pi, which interprets the voice commands received via the microphone. This

interpretation is achieved through advanced speech-to-text conversion, utilizing Speech Recognition and keyword spotting algorithms to accurately process user instructions.

The Arduino Nano functions as the primary controller within the system. It processes the commands and sensor data, subsequently dispatching control signals to the Driver or Application layer. This ensures that user commands are executed efficiently and accurately.

A critical component of the system is the Communication layer, which acts as the intermediary between the different system layers. It facilitates the exchange of data using various communication protocols, including I2C, Serial, and Wi-Fi, ensuring robust and reliable data transmission.

The Sensing layer is equipped with GPS for real-time location tracking, and accelerometer and gyroscope sensors for monitoring the wheelchair's orientation and movement. These sensors play a vital role in ensuring the safety and proper functioning of the wheelchair.

The Driver layer is responsible for executing the movement control commands received from the main controller and managing GSM-based SMS alerts in emergency scenarios. This layer ensures that the wheelchair responds accurately to user inputs and that caregivers are promptly notified in case of emergencies.

The entire system is powered by three Li-ion batteries connected in series, providing reliable power to all components. This comprehensive architecture not only integrates multiple control methods but also offers robust command processing and reliable emergency alerting capabilities, significantly enhancing the functionality and safety of the wheelchair system.

Figures 20 and 21, provided below, illustrate the detailed workflow of the wheelchair, showcasing the seamless integration of its various components and functionalities.

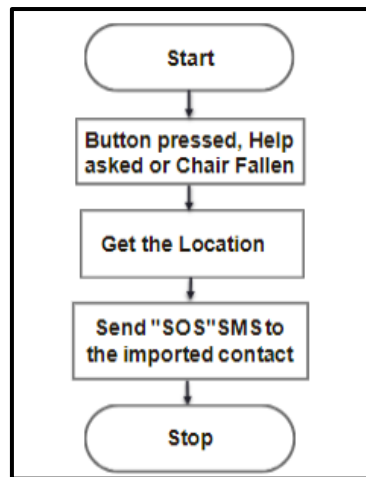


Figure 20 : Flow Chart of the Emergency Alert System

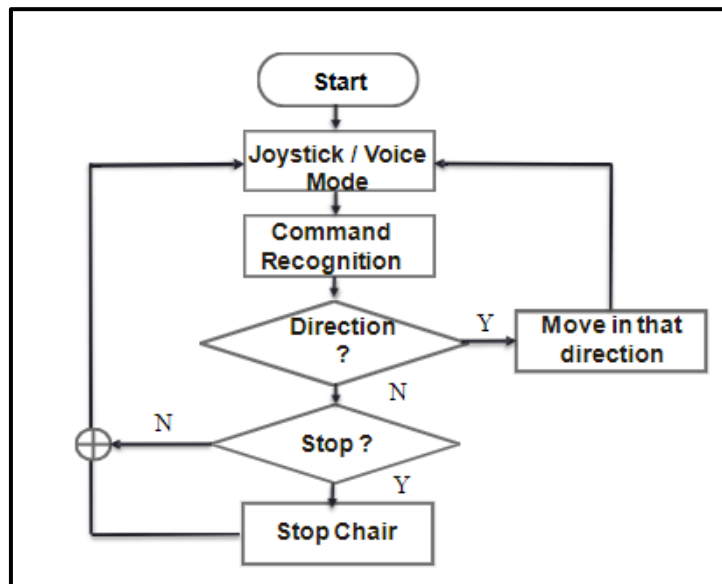


Figure 21 : Flow Chart of Movement Control

4.2 CIRCUIT DESIGN

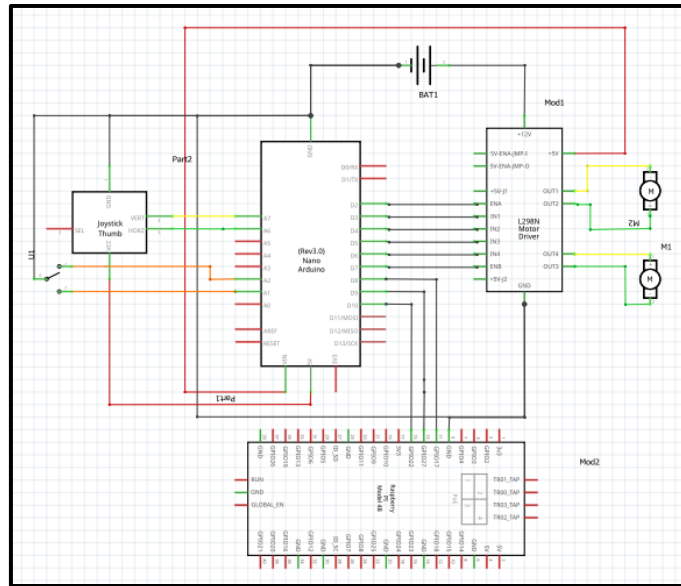


Figure 22 : Schematic Diagram for Movement Control

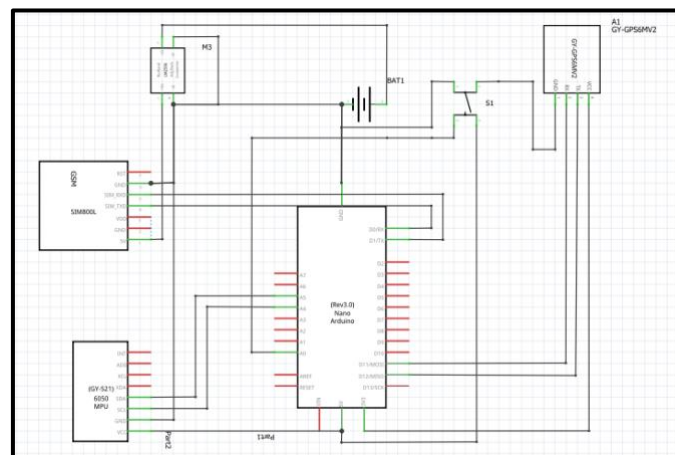


Figure 23 : Schematic Diagram for Emergency Features

The schematic diagrams in figure 22 and figure 23 show the circuit for the movement control and emergency alert system. The superposition of both the circuits gives us information of the overall circuit.

4.3 CIRCUIT IMPLEMENTATION

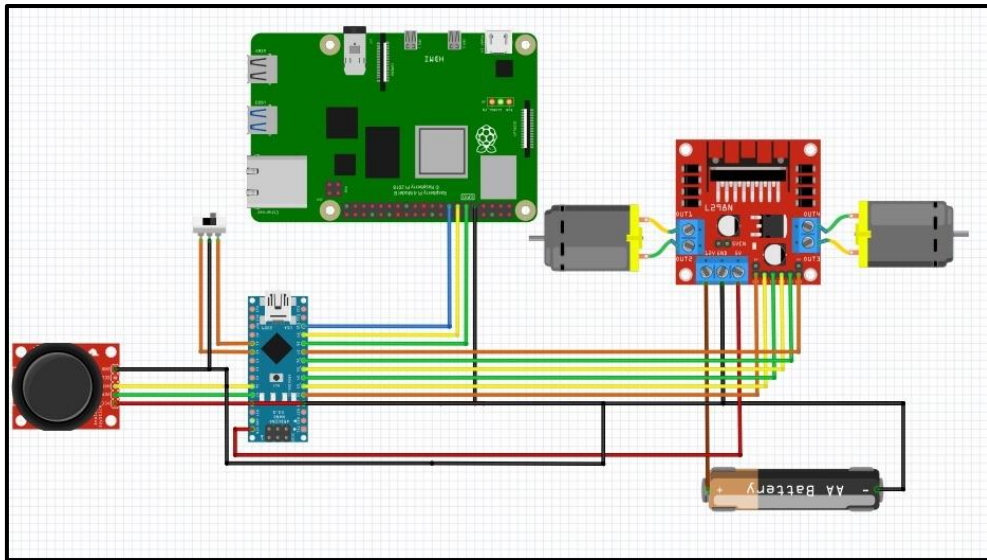


Figure 24 : Circuit Implementation of Movement Control

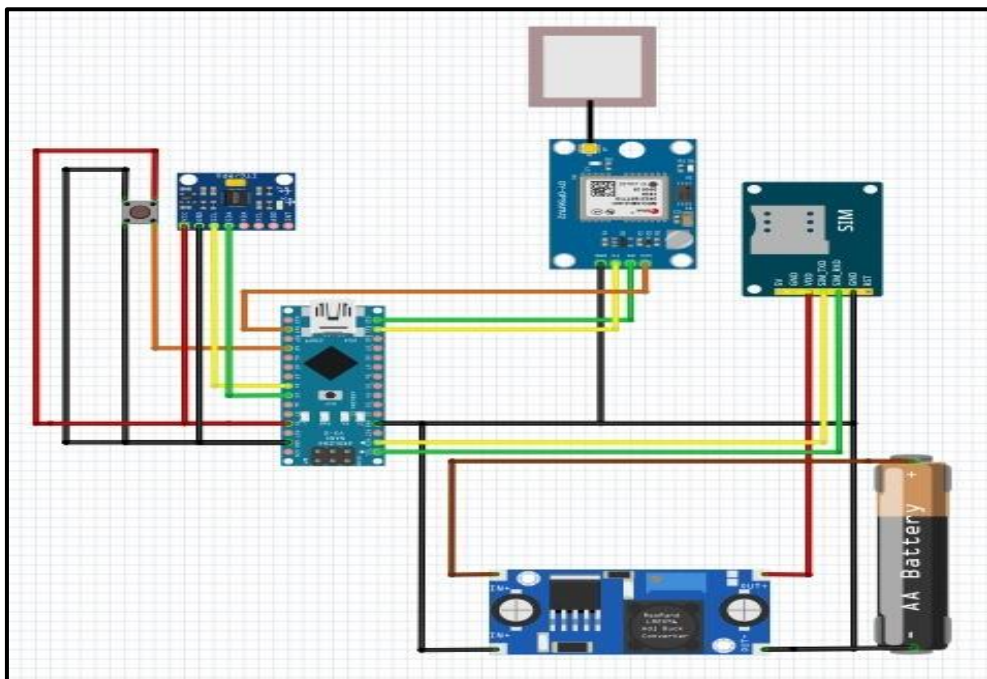


Figure 25 : Circuit Implementation of Emergency Control and Location Tracking

The above breadboard view of the circuit mimics connections of the actual circuit made on hardware. The superposition of the 2 gives us the complete circuit.

CHAPTER-5

RESULT, CONCLUSION & FUTURE SCOPE

5.1 RESULT

We have successfully implemented the wheelchair. The user can operate the wheelchair through voice command and joystick. There is a switch to toggle between the modes of operation. In voice command mode, the user can operate the wheelchair simply using the keywords left, right, forward, backward, turn around, and stop. He/she can also use Hindi keywords like aage, peeche, ruko.

In joystick mode, users who can move their hand can use a joystick to point the direction of the wheelchair. Therefore, these two modes of operation increase accessibility and provide motion independence.



Figure 26 : Final Prototype Image

Should an unforeseen event occur where the wheelchair experiences a collision or accidentally tips over while in operation, an immediate alert is triggered to notify the

designated caregiver. This alert includes vital information such as the precise location where the incident took place, ensuring prompt assistance can be provided to the user.

Furthermore, to facilitate monitoring and ensure the safety of the wheelchair user, caregivers are empowered with the ability to access updates regarding the wheelchair's location. By simply sending a predefined message, such as 'send location', via the Global System for Mobile Communications (GSM), caregivers can receive instant updates on the current whereabouts of the wheelchair.

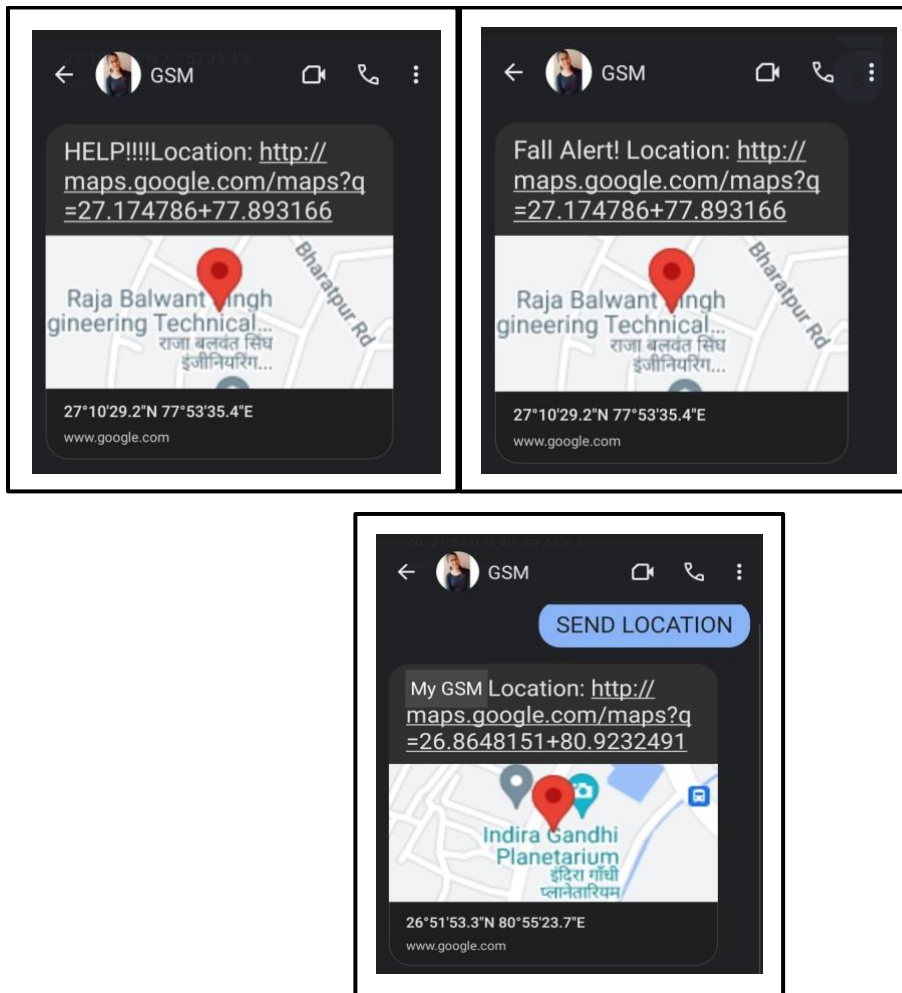


Figure 27 : Screenshots of (a) Help Feature, (b) Fall Alert Feature and (c) Location Tracking

5.2 CONCLUSION

The wheelchair prototype presented here represents a significant leap forward in facilitating independent mobility for individuals with disabilities, particularly those

with locomotive syndrome. By seamlessly integrating multiple modes of control, including voice commands in both English and Hindi, as well as joystick control, the system caters to a diverse range of users, ensuring accessibility regardless of language preference or physical ability. This versatility empowers users to navigate their surroundings with ease, promoting a greater sense of independence and autonomy in their daily lives.

One of the standout features of this wheelchair prototype is its emergency alerting capability, which serves as a vital safety mechanism for users. Equipped with advanced sensors such as accelerometers and gyro sensors, the wheelchair can detect accidents or falls in real-time. In the event of such an incident, the system automatically triggers an alert message containing the wheelchair's precise location, providing caregivers with immediate notification and enabling them to respond swiftly to the user's needs. This proactive approach to safety helps mitigate risks and ensures timely assistance in critical situations, thereby enhancing overall peace of mind for both users and caregivers.

Moreover, the inclusion of a help button further reinforces the system's commitment to user safety and support. With a simple press of the button or a voice command, users can quickly summon assistance when faced with challenges or emergencies. This intuitive feature streamlines the process of seeking help, allowing users to communicate their needs effectively and efficiently, even in situations where verbal communication may be difficult or impossible.

Additionally, the wheelchair's GPS location tracking functionality adds another layer of security and reassurance for caregivers. By providing monitoring of the wheelchair's position, caregivers can always stay informed about the user's whereabouts, facilitating proactive interventions and ensuring prompt responses to unforeseen circumstances. This visibility not only enhances safety but also fosters a greater sense of trust and confidence in the reliability of the system, contributing to a more positive and empowering user experience.

5.3 FUTURE SCOPE

1. **Offline Voice Recognition Implementation:** One significant area for future research is the implementation of voice recognition using offline machine learning (ML) and natural language processing (NLP) models. This would enable the system to function effectively in environments where network availability is limited or weak. Developing robust offline capabilities can ensure continuous operation and improve the system's accessibility in remote or underserved areas.
2. **Enhanced Safety Features:** Implementing advanced safety features is crucial for future development. This could include automatic braking systems to prevent collisions, seat belts with sensors to ensure they are properly fastened, and alerts for tipping risks. Additionally, integrating emergency response systems that notify caregivers or medical professionals in case of an accident could provide an added layer of security for users.
3. **Speed Control Features:** Future research could focus on developing intelligent speed control mechanisms. This might involve adaptive speed control based on terrain and environmental conditions, allowing for safe navigation over different surfaces. Implementing user-specific speed settings and automated deceleration in high-risk areas could enhance both safety and user comfort.

REFERENCES

- [1] Agarwal, R., Siddiqui, A., Singh, K., Solanki, A. S., & Gautam, L. (2016). A Voice Controlled Wheelchair Prototype for a Medically Challenged. *International Journal of Engineering Trends and Technology*. <https://doi.org/10.14445/22315381/ijett-v34p211>
- [2] Arai, K., & Mardiyanto, R. (2011). Eyes Based Electric Wheelchair Control System- - I (eye) can control Electric Wheelchair -. *International Journal of Advanced Computer Science and Applications*, 2(12). <https://doi.org/10.14569/ijacsa.2011.021215>
- [3] Bourhis, G., Moumen, K., Pino, P., Rohmer, S., & Pruski, A. (2002). Assisted navigation for a powered wheelchair. *Proceedings of IEEE Systems Man and Cybernetics Conference - SMC*. <https://doi.org/10.1109/icsmc.1993.385072>
- [4] Cui, J., Huang, Z., Li, X., Cui, L., Shang, Y. & Tong, L. Research on Intelligent Wheelchair Attitude-Based Adjustment Method Based on Action Intention Recognition. *Micromachines* 2023, 14, 1265. <https://doi.org/10.3390/mi14061265>
- [5] Hameed, S., Shoukat, MH., Khan, AR. & Haroon, KM (2021) Smart wheelchair. *Ann Robot Automation* 5(1): 011-018. DOI: <https://dx.doi.org/10.17352/ara.000008>
- [6] Kalantri, R.A. & Chitre, D.K., (2013). Automatic Wheelchair Using Gesture Recognition. *International Journal of Engineering and Advanced Technology*. Volume 2, Issue 6.
- [7] Kaur, A. (2021). Wheelchair control for disabled patients using EMG/EOG based human machine interface: A review. *J. Med. Eng. Technol.* 45, 61–74. <https://doi.org/10.1080/03091902.2020.185383>
- [8] Leaman, J., La, H.M. & Nguyen, L., (2016) Development of a smart wheelchair for people with disabilities. In *Proceedings of the 2016 IEEE International Conference on Multisensor Fusion and Integration for Intelligent Systems (MFI)*, Baden-Baden, Germany, pp. 279–284.
- [9] Meena, K., Gupta, S. & Khare, V. (2017). Voice controlled wheelchair. *International Journal of Electronics, Electrical and Computational System IJEECS* ISSN 2348-117 Volume 6, Issue 4.
- [10] Ning, M., Yu, K., Zhang, C., Wu, Z. & Wang, Y., (2021) Wheelchair design with variable posture adjustment and obstacle-overcoming ability. *J. Braz. Soc. Mech. Sci. Eng.* 43, 197.

- [11] Rahimunnisa, K., Atchaiya, M., Arunachalam, B. and Divyaa, V. (2020). AI-based smart and intelligent wheelchair. *Journal of Applied Research and Technology*. <https://doi.org/0.22201/icat.24486736e.2020.18.6.1351>
- [12] Rakib, M.A., Uddin, S., Rahman, M.M., Chakraborty, S. & Abbas, F.I. (2021) Smart Wheelchair with Voice Control for Physically Challenged People. *Eur. J. Eng. Technol. Res.* 6, 97–102.
- [13] Renuka, K., Harini, R., Balaji, V. and Ashok, N. (2020). Raspberry Pi based Multi-optional Wireless Wheelchair Control and Gesture Recognized Home Assist System. IOP publishing ICCSS. <https://doi.org/10.1088/1757-899X/1084/1/012071>
- [14] Ruzaij, M.F., Neubert, S., Stoll, N. & Thurow, K., (2017) Design and implementation of low-cost intelligent wheelchair controller for quadriplegias and paralysis patients. In *Proceedings of the 2017 IEEE 15th International Symposium on Applied Machine Intelligence and Informatics (SAMi)*, pp. 000399–000404.
- [15] Sangeetha, S., Vigneshwari, S., Dhinakaran, R., Prabu, M. and Karpagam, R. (2019). Voice Controlled Autonomous Wheelchair Using IoT. *International Journal of Research in Engineering, Science and Management* Volume-2, Issue-10.
- [16] Sibai, M.H. & Manap, S.A. (2015). A Study on Smart Wheelchair System. *International Journal of Engineering Technology and Science*. Volume 4. <http://dx.doi.org/10.15282/ijets.4.2015.1.4.1033>
- [17] Simpson R.C, LoPresti EF, Hayashi S, Nourbakhsh I.R, Miller D.P., “The smart wheelchair component system.” *J Rehabil Res Dev*. 2004; 41(3B):429-42.\
- [18] Sukerkar, K., Suratwala, D., Saravade, A., Patil, J. & Britto, R. (2018). Smart Wheelchair. *International Journal of Informatics and Communication Technology (IJ-ICT)* Vol.7, No.2. <https://doi.org/10.11591/ijict.v7i2.pp63-66>
- [19] Thongpance, N. and Chotikunnnan, P. (2023). Design and Construction of Electric Wheelchair with Mecanum Wheel. *Journal of Robotics and Control (JRC)* Volume 4, Issue 1. <https://doi.org/10.18196/jrc.v4i1.17095>
- [20] Tomari, M.R.M., Kobayashi, Y. & Kuno, Y. (2012). Development of Smart Wheelchair System for a User with Severe Motor Impairment. *Procedia Eng.* 41, 538–546
- [21] Wada M. and Kameda F. (2022). A Joystick Car Drive System with Seating in a Wheelchair. DOI: 10.1109/IECON.2009.5415364 · Source: IEEE Xplore <https://doi.org/10.1109/IECON.2009.5415364>

- [22] World Health Organization. Spinal Cord Injury. [Oct; 2022]. 2013.
<https://www.who.int/news-room/fact-sheets/detail/spinal-cord-injury>

APPENDIX

RASPBERRY PI CODE

```
import RPi.GPIO as GPIO

import speech_recognition as sr

# GPIO

GPIO.setwarnings(False)

GPIO.setmode(GPIO.BOARD)

# GPIO.setup(8, GPIO.OUT, initial=GPIO.LOW)


p1 = 17

p2 = 27

p3 = 22


# Set up pin modes

GPIO.setup(p1, GPIO.OUT)

GPIO.setup(p2, GPIO.OUT)

GPIO.setup(p3, GPIO.OUT)


r = sr.Recognizer()

mic = sr.Microphone()
```

```
def stop():
```

```
    GPIO.output(p1, GPIO.LOW)
```

```
    GPIO.output(p2, GPIO.LOW)
```

```
    GPIO.output(p3, GPIO.LOW)
```

```
    print('Stop ')
```

```
def forward():
```

```
    GPIO.output(p1, GPIO.LOW)
```

```
    GPIO.output(p2, GPIO.LOW)
```

```
    GPIO.output(p3, GPIO.HIGH)
```

```
    print('Forward')
```

```
def backward():
```

```
    GPIO.output(p1, GPIO.LOW)
```

```
    GPIO.output(p2, GPIO.HIGH)
```

```
    GPIO.output(p3, GPIO.LOW)
```

```
    print('Backward')
```

```
def left():
```

```
    GPIO.output(p1, GPIO.LOW)
```

```

GPIO.output(p2, GPIO.HIGH)

GPIO.output(p3, GPIO.HIGH)

print('Left ')


def right():

    GPIO.output(p1, GPIO.HIGH)

    GPIO.output(p2, GPIO.LOW)

    GPIO.output(p3, GPIO.LOW)

    print('Right')


def help():

    GPIO.output(p1, GPIO.HIGH)

    GPIO.output(p2, GPIO.LOW)

    GPIO.output(p3, GPIO.HIGH)

    print('help')


def command_detect(transcript):

    transcript = transcript.lower()

    if any(word in transcript for word in ['help']):

        help()

```

```
    return

elif any(word in transcript for word in ['stop']):

    stop()

    return

elif any(word in transcript for word in ['forward', 'go', 'front', 'aage']):

    forward()

    return

elif any(word in transcript for word in ['backward', 'back', 'reverse', 'piche']):

    backward()

    return

elif any(word in transcript for word in ['left', 'ultra', 'ulte']):

    left()

    return

elif any(word in transcript for word in ['right', 'seedha']):

    right()

    return

elif ('turn around' in transcript):

    turn_around()

    return
```

```

while True:

    with mic as source:

        print("Start talking!")

        r.adjust_for_ambient_noise(source)

        audio = r.listen(source)

    try:

        words = r.recognize_google(audio)

        print(words)

        print('\n\n\n\n\n\n\n')

        command_detect(words)

    except sr.UnknownValueError as e:

        print('Unrecognized value')

    except sr.RequestError as e:

        print('Request Error')

    except sr.WaitTimeOut as e:

        print('Wait TimeOut value')

    except sr.RequestTimedOut as e:

        print('Request TimeOut value')

```

ARDUINO CODE

```

#include <Wire.h>

#include <SoftwareSerial.h>

```

```

#include <TinyGPS++.h>

const int baudRate = 9600;

// For GPS

TinyGPSPlus gps;

SoftwareSerial gpsSerial(11, 12); //RX,TX of Arduino

String Lat = "";

String Lng = "";

String GmapsLink = "http://maps.google.com/maps?q=";

// For GSM we will use hardwareSerial

String SOSNum = "+919520829445";

String fromGSM = ""; // Buffer to store data from GSM

#define SOSButton A0

const unsigned long int SOSButtonPeriod = 5000000; //5s = 5*1000000

unsigned long buttonPressTime = 0;           // Variable to store the time button was
pressed

bool buttonPressed = false;                  // Flag to indicate if the button was pressed

// Pick between Voice or Joystick

#define voiceCmd A1

```

```

// Pins for L298N Driver


// Right Motor out12

#define rightSpeed 2 // enA

#define right1 3 // in1

#define right2 4 // in2


// Left Motor out34

#define left1 5 // in3

#define left2 6 // in4

#define leftSpeed 7 // enB


// 3 pins for voice commands (digital)

#define RPiPin1 8 // RPi - 17

#define RPiPin2 9 // RPi - 27

#define RPiPin3 10 // RPi - 22


// To Read RPi Values

uint8_t p1 = 0;

uint8_t p2 = 0;

uint8_t p3 = 0;

```

```

// 3 pins for joystick control (2 analog 1 digital(optional))

#define xAxisPin A6

#define yAxisPin A7


// To Read joystick values

int xAxisData = 0;

int yAxisData = 0;

int rightSpeedVal = 0; //Right

int leftSpeedVal = 0; //Left

int maxSpeed = 200;

int minSpeed = 150;

// For MPU6050

const int MPU = 0x68; // MPU6050 I2C address

float AccX, AccY, AccZ; // linear Acceleration

float GyroX, GyroY, GyroZ; //angular velocity

float accAngleX, accAngleY, gyroAngleX, gyroAngleY, gyroAngleZ; //used in void
loop()

float roll, pitch, yaw;

float AccErrorX, AccErrorY, GyroErrorX, GyroErrorY, GyroErrorZ;

float elapsedTime, currentTime, previousTime;

int c = 0;

float angle; //due to how I orientated my MPU6050 on my car, angle x= roll

```

```

float targetAngle = 0;

int equilibriumSpeed = 248; //rough estimate of PWM at the speed pin of the stronger

bool isDriving = false; //if the car driving forward OR rotate/stationary

bool prevIsDriving = true; //equals isDriving in the previous iteration of void loop()


// For Fall Detection

const unsigned long alertInterval = 1200000000; // 2 min = 2*60*10^6 micros

unsigned long previousAlertTime = 0;

bool fallDetected = false;


// For GSM GPS tracker

String buff;

uint8_t d = 0;

uint8_t i = 0;

uint8_t *buffer1 = 0;

void getLocation() {

    while (gpsSerial.available() > 0) {

        if (gps.encode(gpsSerial.read())) {

            if (gps.location.isValid()) {

                Lat = String(gps.location.lat(), 6);

```

```

        Lng = String(gps.location.lng(), 6);

    }

}

return;

}

void sendSMS(String message) {

    digitalWrite(13,HIGH);

    Serial.print("AT+CMGF=1\r");

    delay(100);

    Serial.println("AT+CMGS=\"" + SOSNum + "\"\r");

    delay(500);

    Serial.print(message);

    Serial.print("Location: ");

    Serial.print(GmapsLink);

    Serial.print(Lat);

    Serial.print("+");

    Serial.print(Lng);

    delay(100);

```

```

Serial.println((char)26);

delay(100);

Serial.println();

delay(100);

digitalWrite(13,LOW);

}

// Check if SOS button is pressed

void checkForButtonPress() {

    // Check if the button is pressed

    if (digitalRead(SOSButton) == LOW) {

        if (!buttonPressed) {

            Serial.println("Button has been pressed");

            buttonPressTime = micros(); // Record the time the button was pressed

            buttonPressed = true;      // Set the button pressed flag

        }

    } else {

        // Button is released

        if (buttonPressed) {

            // Button was previously pressed

            unsigned long pressDuration = micros() - buttonPressTime;

```

```
        if (pressDuration >= SOSButtonPeriod) { // Check if button was pressed for more
        than 5 seconds
```

```
            // Serial.println(pressDuration);
```

```
            // Serial.println(SOSButtonPeriod);
```

```
            sendSMS("HELP!!!!"); // If button was pressed for more than 5 seconds, send
SMS
```

```
        }
```

```
        buttonPressed = false; // Reset button pressed flag
```

```
    }
```

```
}
```

```
}
```

```
// For GPS Tracker
```

```
void checkForSMS(){
```

```
    while(Serial.available()){
```

```
        buff = Serial.readString();
```

```
        // Serial.println(buff);
```

```
        // buffer1 = strstr(buff, "LOCATION");
```

```
        buff.trim();
```

```
        // Serial.println(buff);
```

```
        if (buff == "s"){
```

```
            sendSMS("Location: ");
```

```

    }

}

}

```

```

void calculateAngle() {

    readAcceleration();

    // Calculating Roll and Pitch from the accelerometer data

    accAngleX = (atan(AccY / sqrt(pow(AccX, 2) + pow(AccZ, 2))) * 180 / PI) -
    AccErrorX; //AccErrorX is calculated in the calculateError() function

    accAngleY = (atan(-1 * AccX / sqrt(pow(AccY, 2) + pow(AccZ, 2))) * 180 / PI) -
    AccErrorY;

    // === Read gyroscope (on the MPU6050) data === //

    previousTime = currentTime;

    currentTime = micros();

    elapsedTime = (currentTime - previousTime) / 1000000; // Divide by 1000 to get
seconds

    readGyro();

    // Correct the outputs with the calculated error values

    GyroX -= GyroErrorX; //GyroErrorX is calculated in the calculateError() function

    GyroY -= GyroErrorY;

    GyroZ -= GyroErrorZ;

    // Currently the raw values are in degrees per seconds, deg/s, so we need to multiply
by sendonds (s) to get the angle in degrees

```

```

gyroAngleX += GyroX * elapsedTime; // deg/s * s = deg

gyroAngleY += GyroY * elapsedTime;

yaw += GyroZ * elapsedTime;

//combine accelerometer- and gyro-estimated angle values. 0.96 and 0.04 values are
determined through trial and error by other people

roll = 0.96 * gyroAngleX + 0.04 * accAngleX;

pitch = 0.96 * gyroAngleY + 0.04 * accAngleY;

angle = roll; //if you mounted MPU6050 in a different orientation to me, angle may
not = roll. It can roll, pitch, yaw or minus version of the three

//for me, turning right reduces angle. Turning left increases angle.

}

void calculateError() {

    //When this function is called, ensure the car is stationary. See Step 2 for more info

    // Read accelerometer values 200 times

    c = 0;

    while (c < 200) {

        readAcceleration();

        // Sum all readings

        AccErrorX += (atan((AccY) / sqrt(pow((AccX), 2) + pow((AccZ), 2))) * 180 / PI);

        AccErrorY += (atan(-1 * (AccX) / sqrt(pow((AccY), 2) + pow((AccZ), 2))) * 180 /
PI);

        c++;

```

```

}

//Divide the sum by 200 to get the error value, since expected value of reading is zero

AccErrorX = AccErrorX / 200;

AccErrorY = AccErrorY / 200;

c = 0;

// Read gyro values 200 times

while (c < 200) {

    readGyro();

    // Sum all readings

    GyroErrorX += GyroX;

    GyroErrorY += GyroY;

    GyroErrorZ += GyroZ;

    c++;

}

//Divide the sum by 200 to get the error value

GyroErrorX = GyroErrorX / 200;

GyroErrorY = GyroErrorY / 200;

GyroErrorZ = GyroErrorZ / 200;

// Serial.println("The the gryoscope setting in MPU6050 has been calibrated");

}

```

```

void readAcceleration() {

    Wire.beginTransmission(MPU);

    Wire.write(0x3B); // Start with register 0x3B (ACCEL_XOUT_H)

    Wire.endTransmission(false);

    Wire.requestFrom(MPU, 6, true); // Read 6 registers total, each axis value is stored
in 2 registers

    //For a range of +-2g, we need to divide the raw values by 16384, according to the
MPU6050 datasheet

    AccX = (Wire.read() << 8 | Wire.read()) / 16384.0; // X-axis value

    AccY = (Wire.read() << 8 | Wire.read()) / 16384.0; // Y-axis value

    AccZ = (Wire.read() << 8 | Wire.read()) / 16384.0; // Z-axis value

}

void readGyro() {

    Wire.beginTransmission(MPU);

    Wire.write(0x43);

    Wire.endTransmission(false);

    Wire.requestFrom(MPU, 6, true);

    GyroX = (Wire.read() << 8 | Wire.read()) / 131.0;

    GyroY = (Wire.read() << 8 | Wire.read()) / 131.0;

    GyroZ = (Wire.read() << 8 | Wire.read()) / 131.0;

}

```

```
void forward() {  
  
    // Set right motor forward  
  
    digitalWrite(right1, HIGH);  
  
    digitalWrite(right2, LOW);  
  
    // Set left motor forward  
  
    digitalWrite(left1, HIGH);  
  
    digitalWrite(left2, LOW);  
  
    // Serial.write("Forward\n");  
  
}
```

```
void backward() {  
  
    // Set right motor backward  
  
    digitalWrite(right1, LOW);  
  
    digitalWrite(right2, HIGH);  
  
    // Set left motor backward  
  
    digitalWrite(left1, LOW);  
  
    digitalWrite(left2, HIGH);  
  
    // Serial.write("Backward\n");  
  
}
```

```
void turnRight() {  
  
    // Set right motor backward  
  
    digitalWrite(right1, HIGH);  
  
    digitalWrite(right2, LOW);  
  
    // Set left motor forward  
  
    digitalWrite(left1, LOW);  
  
    digitalWrite(left2, HIGH);  
  
    // Serial.write("Right\n");  
  
}
```

```
void turnLeft() {  
  
    // Set right motor forward  
  
    digitalWrite(right1, LOW);  
  
    digitalWrite(right2, HIGH);  
  
    // Set left motor backward  
  
    digitalWrite(left1, HIGH);  
  
    digitalWrite(left2, LOW);  
  
    // Serial.write("Left\n");  
  
}
```

```
void stop() {
```

```

// Stop right motor

digitalWrite(right1, LOW);

digitalWrite(right2, LOW);

// Stop left motor

digitalWrite(left1, LOW);

digitalWrite(left2, LOW);

// Serial.write("Stop\n");

}

void detectFall() {

  readAcceleration();

  if (AccX < -0.09 || abs(AccY) > 0.4 || abs(AccZ) > 0.6) {

    if (!fallDetected) {

      // Fall detected, send SMS alert

      stop();

      // Serial.print("AccX: ");

      // Serial.print(AccX);

      // Serial.print(" AccY: ");

      // Serial.print(AccY);

      // Serial.print(" AccZ: ");

      // Serial.println(AccZ);

```

```

    // Serial.print("Fall Detected!\n");

    sendSMS("Fall Alert! ");

    fallDetected = true;

    previousAlertTime = currentTime; // Record the time when the alert was sent
} else {

    // Check if it is time to send another alert

    // Serial.print(currentTime - previousAlertTime);

    if (currentTime - previousAlertTime >= alertInterval) {

        // Serial.print("Fall Alert!!!!!!\n");

        sendSMS("Fall Alert!!!!!!");

        previousAlertTime = currentTime; // Update the time for the next alert

    }

}

} else {

    fallDetected = false;

}

if (!fallDetected) {

    previousAlertTime = 0;

}

}

```

```

void voiceControl() {

    p1 = digitalRead(RPiPin1);

    p2 = digitalRead(RPiPin2);

    p3 = digitalRead(RPiPin3);

    calculateAngle();

    if (p1 == 0 && p2 == 0 && p3 == 0) {

        stop();

    } else if (p1 == 0 && p2 == 0 && p3 == 1) {

        forward();

    } else if (p1 == 0 && p2 == 1 && p3 == 0) {

        backward();

    } else if (p1 == 0 && p2 == 1 && p3 == 1) {

        turnRight();

    } else if (p1 == 1 && p2 == 0 && p3 == 0) {

        turnLeft();

    } else if (p1 == 1 && p2 == 0 && p3 == 1) {

        sendSMS("Help");

        stop();

    }

    // Serial.print("p1: ");

```

```

// Serial.print(p1);

// Serial.print(" p2: ");

// Serial.print(p2);

// Serial.print(" p3: ");

// Serial.println(p3);

analogWrite(leftSpeed, maxSpeed);

analogWrite(rightSpeed, maxSpeed);

}

```

```

void joystickControl() {

  xAxisData = analogRead(xAxisPin);

  yAxisData = analogRead(yAxisPin);

  // Serial.print("X : ");

  // Serial.print(xAxisData);

  // Serial.print("Y : ");

  // Serial.println(yAxisData);

  // Direction   X | Y

  // Forward : 1023 | 512

  // Backward : 0 | 512

  // Left : 512 | 0

  // Right : 512 | 1023

```

```

// Stop : 512 | 512

if (fallDetected || (xAxisData > 470 && xAxisData < 550 && yAxisData > 470 &&
yAxisData < 550)) {

    stop();

}

else if (xAxisData > 470 && xAxisData < 550) {

    if (yAxisData < 470) {

        turnRight();

        // Convert the declining Y-axis readings for going backward from 470 to 0 into 0 to
        255 value for the PWM signal for increasing the motor speed

        rightSpeedVal = map(yAxisData, 470, 0, 0, 255);

        leftSpeedVal = map(yAxisData, 470, 0, 0, 255);

    }

    else if (yAxisData > 550) {

        turnLeft();

        // Convert the increasing Y-axis readings for going forward from 550 to 1023 into
        0 to 255 value for the PWM signal for increasing the motor speed

        rightSpeedVal = map(yAxisData, 550, 1023, 0, 255);

        leftSpeedVal = map(yAxisData, 550, 1023, 0, 255);

    }

}

else {

    if (yAxisData > 470 && yAxisData < 550) {

```

```

if (xAxisData > 550) {

    forward();

    // Convert the increasing X-axis readings for going forward from 550 to 1023 into
    0 to 255 value for the PWM signal for increasing the motor speed

    rightSpeedVal = map(xAxisData, 550, 1023, 0, 255);

    leftSpeedVal = map(xAxisData, 550, 1023, 0, 255);

}

if (xAxisData < 470) {

    backward();

    // Convert the declining X-axis readings for going backward from 470 to 0 into 0
    to 255 value for the PWM signal for increasing the motor speed

    rightSpeedVal = map(xAxisData, 470, 0, 0, 255);

    leftSpeedVal = map(xAxisData, 470, 0, 0, 255);

}

} else {

    if (xAxisData > 550) {

        forward();

        // Convert the increasing Y-axis readings from 550 to 1023 into 0 to 255 value

        int yMapped = map(yAxisData, 550, 1023, 0, 250);

        // Move right - decrease right motor speed, increase left motor speed

```

```

    rightSpeedVal = rightSpeedVal - yMapped;

    leftSpeedVal = leftSpeedVal + yMapped;

    // Confine the range from 0 to 255

    if (rightSpeedVal < 70) { rightSpeedVal = 70; }

    if (leftSpeedVal > 255) { leftSpeedVal = 255; }

}

if (xAxisData < 470) {

    backward();

    // Convert the declining Y-axis readings from 470 to 0 into increasing 0 to 255
value

    int yMapped = map(yAxisData, 470, 0, 0, 255);

    // Move to left - decrease left motor speed, increase right motor speed

    rightSpeedVal = rightSpeedVal + yMapped;

    leftSpeedVal = leftSpeedVal - yMapped;

    // Confine the range from 0 to 255

    if (rightSpeedVal > 255) { rightSpeedVal = 255; }

    if (leftSpeedVal > 70) { leftSpeedVal = 70; }

}

}

}

// Prevent buzzing at low speeds (Adjustment according to motors. Motors could not
start moving if PWM value was below value of 70)

```

```

if (rightSpeedVal < 70) { rightSpeedVal = 0; }

if (leftSpeedVal < 70) { leftSpeedVal = 0; }


rightSpeedVal = min(maxSpeed, rightSpeedVal);

leftSpeedVal = min(maxSpeed, leftSpeedVal);

analogWrite(rightSpeed, rightSpeedVal); // Send PWM signal to motor A

analogWrite(leftSpeed, leftSpeedVal); // Send PWM signal to motor B

}


void setUpPinModes() {

  pinMode(voiceCmd, INPUT_PULLUP);


  pinMode(SOSButton, INPUT);

  pinMode(RPiPin1, INPUT);

  pinMode(RPiPin2, INPUT);

  pinMode(RPiPin3, INPUT);

  pinMode(xAxisPin, INPUT);

  pinMode(yAxisPin, INPUT);

  pinMode(rightSpeed, OUTPUT);

  pinMode(right1, OUTPUT);

  pinMode(right2, OUTPUT);

```

```

pinMode(leftSpeed, OUTPUT);

pinMode(left1, OUTPUT);

pinMode(left2, OUTPUT);

pinMode(13, OUTPUT);
}

void setup() {

digitalWrite(13, HIGH);

Serial.begin(baudRate);

gpsSerial.begin(baudRate);

Wire.begin();

Wire.beginTransmission(MPU); // Start communication with MPU6050 // MPU=0x68

Wire.write(0x6B);           // Talk to the register 6B

Wire.write(0x00);           // Make reset - place a 0 into the 6B register

Wire.endTransmission(true); //end the transmission

// Call this function if you need to get the IMU error values for your module

calculateError();

delay(20);

setUpPinModes();

delay(1000);

```

```

currentTime = micros();

digitalWrite(13, LOW);

}

void loop() {

  if (!digitalRead(voiceCmd)) {

    voiceControl();

  } else {

    // Serial.print("HERE");

    joystickControl();

  }

  // Continuously track location

  getLocation();

  // Check if help is asked via button Press

  checkForButtonPress();

  // For Sending Location GPS Tracker

  checkForSMS();

  // Check if chair has fallen

  detectFall();

  delay(100);

}

```